SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

Mode: In Class

Total Marks:50

Hours : 60 mts

Annexure A

ANALYTICAL PROBLEM SOLVING

89

Code of Conduct:

I K. Yogamand *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/4	/4	/4	/4	/4	
2	/4	/4	/4	/4	/4	
3	/4	/4	/4	/4	/4	
4	/4	/4	/4	/4	/4	
5	/4	/4	/4	/4	/4	
OVERALL RUBRICS VALUE						
	/4	/4	/4	/4	/4	
	/20	/20	/25	/20	/15	/100

CO1 AT1

Compiler Design

K. Yogananda
192525173

1. The Given expression is

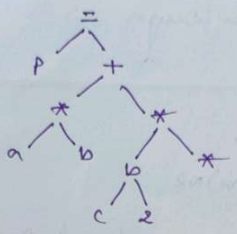
$$p = (a \times b) + (c \times d) \times 2$$

(a) Tokens Generated during lexical Analysis :-

Lexeme	Token
p	Identifier
=	Assignment operator
(	Left parenthesis
a	Identifier
*	Multiplication operator
b	Identifier
)	Right parenthesis
+	Addition Operator
(	Identifier
2	Integer Constant

(b) Syntax Tree :-

- the expression is interpreted according to the operator precedence.



(c) Intermediate Code:-

$t_1 = a * b$

$t_2 = b * c$

$t_3 = t_2 * 2$

$t_4 = t_1 + t_3$

$p = t_4$

(d) Optimization:-

$t_1 = a * b$

$t_2 = b * c$

$t_3 = t_2 * 2$

$p = t_1 + t_3$

(e) Significance of final target Code:-

```
LOAD R1, a
MUL R1, b
LOAD R2, b
MUL R2, c
MUL R2, 2
ADD R1, R2
STORE p, R1
```

Construct the language and write five valid strings:-

(a) $(\varepsilon + a a^*)^*$

* ε = empty string
* a^* = zero or more a
* therefore the expression generates strings containing any of

Language:-

$$L = \{a^n / n \geq 0\}$$

five valid strings:-

ε
a
a a
a a a
a a a a

(b) $(a^4)^* a b b$

the expression must end with

abb

the first part can generate strings made from as and bs

five valid strings:-

a b b
a a b b
b a b b
a b a b b
a a b a b b

(c) $(a^*)^* + b^*)1^*$

- the fixed middle portion is;
  aba
- the expression before and after it can generate combi of a and b's

five valid strings:

a b a
a a b a
b a b a
a b a a
a b a b

(d) $(a + ab)^*$

- each repeated unit is either
  a
  or
  ab

five valid strings

ε
a
a b
a a
a a b

(e) $(abaa)^1 * + (a^*)^1 *$

this is a union of two long values

first part:

$(abaa)^*$

generates :-

ε

a b a

a b a a b a

a b a a b a a b a

second proof :-

$(a*b*)^*$

Generates strings made from blocks of as followed by bs

Few valid strings :-

ε

a b a

a a

b b

a a b b

3 Find the number of Tokens :-

Given c statement is ;

```
main()
{   int a=10;
    char b = "abc";
    int c=30;
    char d = "**++**";
    in /* Comment */ m = 110.5;
}
```

Important Rule :

white space separates Tokens, and the Comment

/* comment */

is ignored by the lexical Analyser

```
line 1
main()
Tokens;
main
( )
{number = 3
line 2 : { ; 4. {
line 3 : int a=10 ; $ int b_a 7 = 8.10.9 ;
line 4 ; char b = "abc" ; : char, b, = "abc" ; ;
line 5 ; int c=30 ; in, t, c, = 30 ;
line 6 ; char d = "xyz" ; ch, ar, d, = "xyz" ; ;
line 7 ; in /* comment */ m = 40.5 int m, = 40.5 ;
final line } : }
```

(1) Identify lexical errors :-

Given statement :

```
void main()
{
    int x = 10, y = 20 ;
    char a ;
    a = $x ;
    x = 2 x ab ;
}
```

Error

the statement is ;

```
x = 1xab ;
```

1xab is an invalid integer literal because an identifier cannot start with a digit

It is being interpreted as Correct type :

1 * x ab,

must there is no valid operator b/w them
correct forms:
if the intentions is an integer:
x = 1;
if the intentions is an integer:
x = 1;
if the intentions is hexa decimal value
x = 0x1b;
so the main lexical error is:
1x als → invalid numeric constant

4 Regular Expression :-

Assume the alphabets is
$\Sigma = \{0,1\}$

(i) string having an even number of 0's
Regular expression
$(1^* (01^*01^*)^*$

Ex:
1 1
0 0
1 0 0 1
0 1 0 1 0
1 1 0 0 1 1

(ii) Strings containing at least one
$a^* b^* (ab^*)^*$

Ex:-
1
0 1
1 0

1 0 1
0 0 0 1

(iii) Both number of as & no. of bs are odds
regular expression is
0 (00)* (11)* + 1 (11)* 0 (00)*

ex: 0 1
1 0
0 0 0 1
1 0 0 1
0 1 1 1
1 1 1 0

(iv) string state that start with 1 and end with 0
1 (0+1)* 0

ex:
1 0
1 0 0
1 1 0
1 0 1 0
1 1 1 0

5 Lexical Analyzer:-

Given
int var123 = 12345; float var456 = 123.45;
if (var123 == var456) { return; }

(i) Given expression contains = 20 tokens

(ii) key words:
int
float total = 4
if

operators: 5 Tokens

separators: 4 Tokens

identifiers: 4 Tokens

integer literals = 12345 = 1 Token

Total = 4+5+4+4+1+1 = 20 Tokens

(iii) FSM state transitions:

int

start → ; → n → t → Accept

similarly ints

float

if

return

identifiers;

var 123

start: → v → a → r → 1 → 2 → 3 → Accept

integer: 12345

start → 1 → 2 → 3 → 4 → 5 → Accept

float pi = 12345

Operators

= =

start → = → = → Accept

Separators:

such as;

() {};

Total Tokens = 20 //